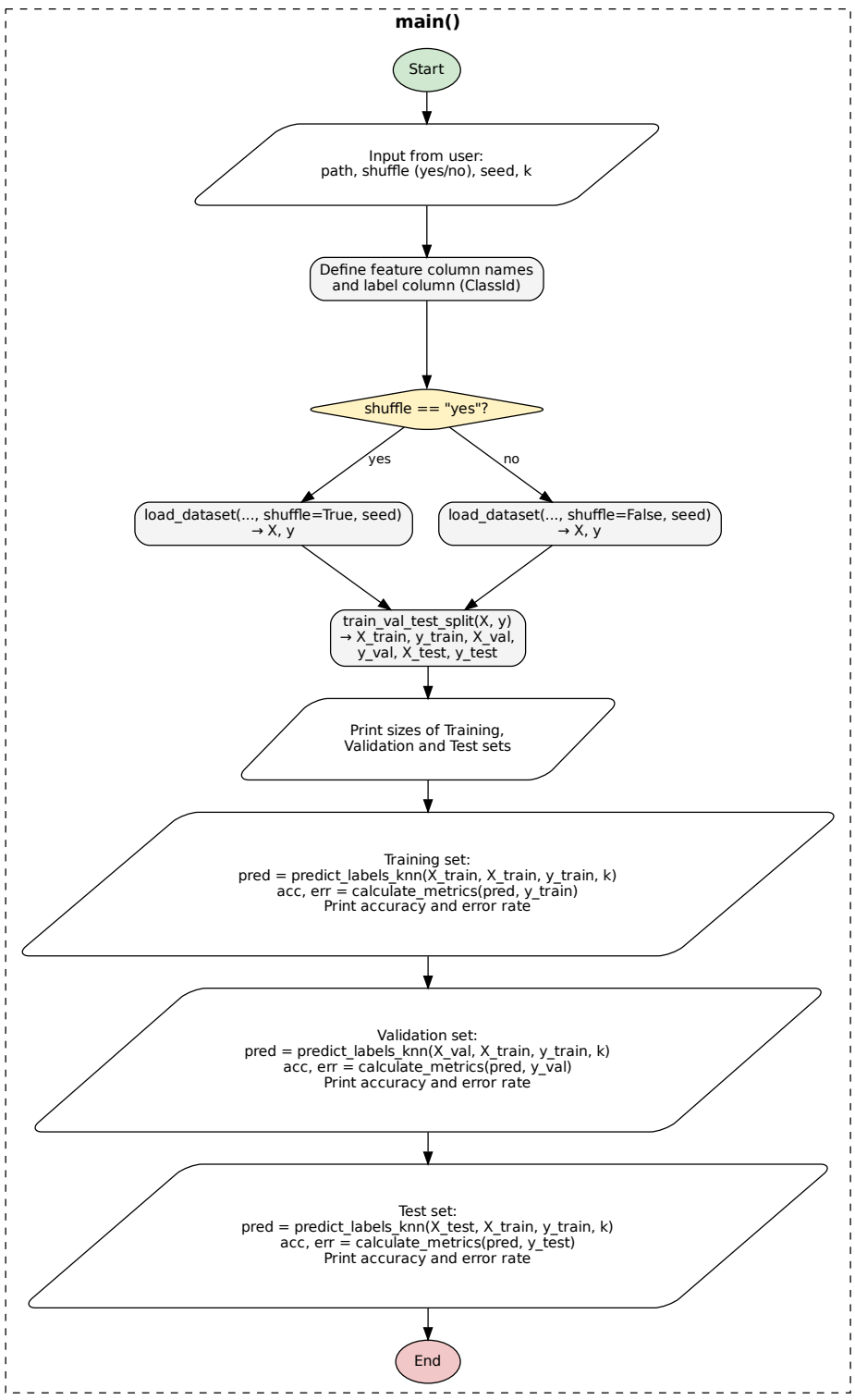
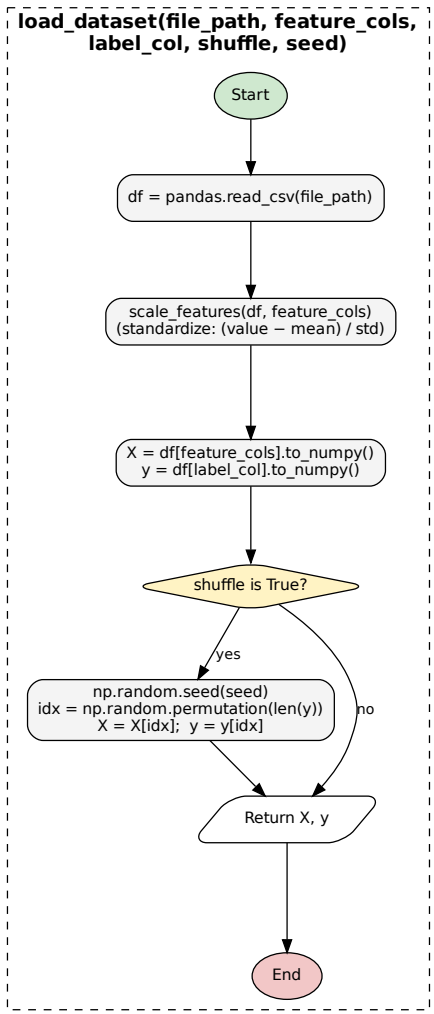
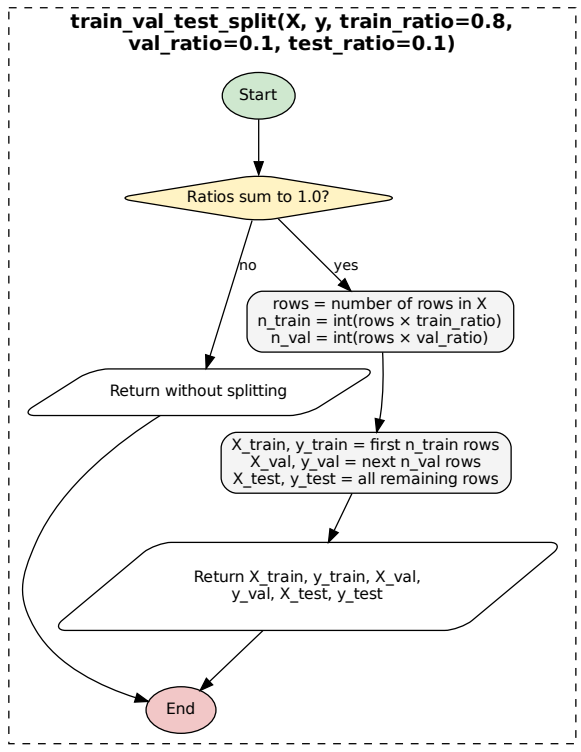
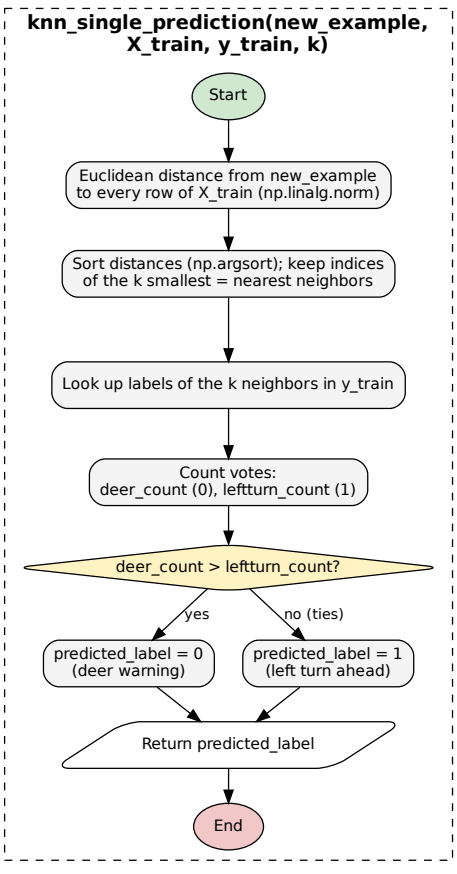
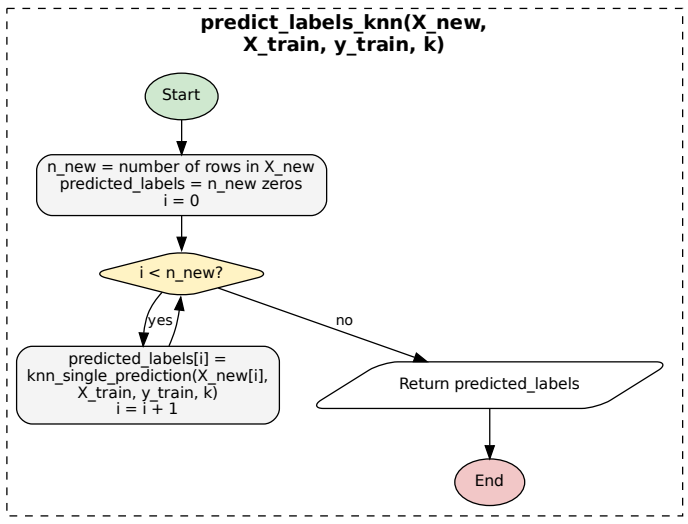
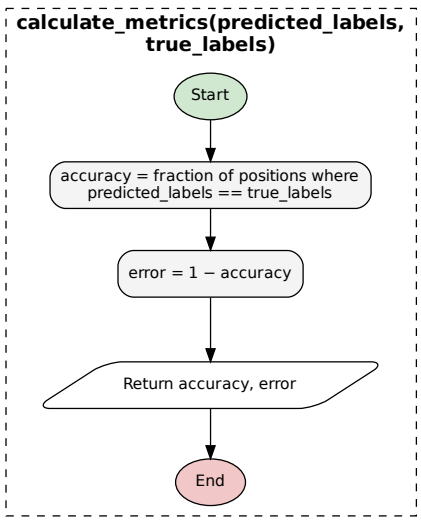

main()
Start
Input from user:
path, shuffle (yes/no), seed, k
Define feature column names
and label column (ClassId)
shuffle == "yes"?
yes
no
load_dataset(..., shuffle=True, seed)
→ X, y
load_dataset(..., shuffle=False, seed)
→ X, y
train_val_test_split(X, y)
→ X_train, y_train, X_val,
y_val, X_test, y_test
Print sizes of Training,
Validation and Test sets
Training set:
pred = predict_labels_knn(X_train, X_train, y_train, k)
acc, err = calculate_metrics(pred, y_train)
Print accuracy and error rate
Validation set:
pred = predict_labels_knn(X_val, X_train, y_train, k)
acc, err = calculate_metrics(pred, y_val)
Print accuracy and error rate
Test set:
pred = predict_labels_knn(X_test, X_train, y_train, k)
acc, err = calculate_metrics(pred, y_test)
Print accuracy and error rate
End


load_dataset(file_path, feature_cols,
label_col, shuffle, seed)
Start
df = pandas.read_csv(file_path)
scale_features(df, feature_cols)
(standardize: (value − mean) / std)
X = df[feature_cols].to_numpy()
y = df[label_col].to_numpy()
shuffle is True?
yes
no
np.random.seed(seed)
idx = np.random.permutation(len(y))
X = X[idx]; y = y[idx]
Return X, y
End


train_val_test_split(X, y, train_ratio=0.8,
val_ratio=0.1, test_ratio=0.1)
Start
Ratios sum to 1.0?
no
yes
rows = number of rows in X
n_train = int(rows × train_ratio)
n_val = int(rows × val_ratio)
Return without splitting
X_train, y_train = first n_train rows
X_val, y_val = next n_val rows
X_test, y_test = all remaining rows
Return X_train, y_train, X_val,
y_val, X_test, y_test
End


knn_single_prediction(new_example,
X_train, y_train, k)
Start
Euclidean distance from new_example
to every row of X_train (np.linalg.norm)
Sort distances (np.argsort); keep indices
of the k smallest = nearest neighbors
Look up labels of the k neighbors in y_train
Count votes:
deer_count (0), leftturn_count (1)
deer_count > leftturn_count?
yes
no (ties)
predicted_label = 0
(deer warning)
predicted_label = 1
(left turn ahead)
Return predicted_label
End


predict_labels_knn(X_new,
X_train, y_train, k)
Start
n_new = number of rows in X_new
predicted_labels = n_new zeros
i = 0
i < n_new?
yes
no
predicted_labels[i] =
knn_single_prediction(X_new[i],
X_train, y_train, k)
i = i + 1
Return predicted_labels
End


calculate_metrics(predicted_labels,
true_labels)
Start
accuracy = fraction of positions where
predicted_labels == true_labels
error = 1 − accuracy
Return accuracy, error
End